

Professional Summary:

- Results-driven **Full Stack Developer** with 7+ years of experience designing, developing, and maintaining enterprise-grade financial applications using **Java, Spring Boot, AWS, and modern microservices architectures**.
- Strong expertise in **Spring Cloud Netflix OSS (Eureka, Ribbon, Hystrix), Kafka, and Dockerized microservices**, ensuring high scalability, fault tolerance, and seamless communication across distributed systems.
- Built modern, responsive **Single Page Applications (SPAs)** using **React.js, Angular, and Vue.js**, improving page load time, interactivity, and customer engagement across platforms.
- Designed and developed **Java-based REST APIs and event-driven microservices** using **Spring Boot and Kafka**, improving backend efficiency, reducing latency, and enabling scalable concurrent processing.
- Deployed and supported **Java services on AWS and on-prem Tomcat servers**, enabling hybrid architecture and achieving **zero-downtime releases** across cloud and local environments.
- Implemented **microservice and service-oriented architecture (SOA) patterns**, standardizing contracts, error handling, and versioning for reusable and maintainable service integration.
- Built containerized services using **Docker**, improving environment parity, reducing cold start times, and simplifying deployments across Dev, QA, and Production pipelines.
- Owned **end-to-end DevSecOps automation in CI/CD (Jenkins, Gradle)** with integrated **unit, component, functional, performance, and security testing** stages, ensuring code quality and compliance before production deployment.
- Created **automation and service virtualization frameworks** using **Mockito, WireMock, and RestAssured**, enabling parallel testing, defect isolation, and continuous regression coverage.
- Worked hands-on with **SQL and NoSQL databases** including **Oracle, PL/SQL, SQL Server, and DynamoDB**, optimizing schema design, query performance, and data consistency across hybrid data systems.
- Produced and maintained **architecture and design artifacts** (UML class and sequence diagrams, Architecture Decision Records), and validated implementation through **performance (JMeter), security (OAuth2, SAST scans), and operability checks (CloudWatch, Splunk)** to ensure design compliance.
- Skilled in writing clean, test-driven, and well-documented code following **Agile and CI/CD best practices**, improving team velocity and release stability.
- Developed and optimized backend **RESTful APIs** using **Spring Boot, Spring Data JPA, and Hibernate**, providing a robust communication layer between front-end modules and databases for real-time transactions.
- Converted outdated **SOAP web services** into modern **Spring Boot REST APIs**, improving maintainability, response times, and simplifying client consumption.
- Created reusable **UI components in React.js** with features such as filtering, pagination, and editable tables, increasing development efficiency and consistency across projects.
- Designed **microservices architectures** using **Spring Cloud Netflix OSS (Eureka, Zuul, Hystrix)**, enabling dynamic scaling, service self-registration, and resilient failure recovery.
- Integrated **OAuth2 and OpenID Connect** into backend microservices for secure authentication and authorization across distributed systems.
- Validated APIs using **Postman and SoapUI**, creating automated test suites to ensure endpoint integrity, schema correctness, and adherence to business rules.
- Automated **build, deployment, and test workflows** using **Jenkins, Gradle, and SonarQube**, incorporating code coverage metrics, quality gates, and notification pipelines to improve reliability and reduce manual intervention.
- **Containerized applications with Docker** to ensure consistent configuration across developer machines, staging, and production environments, simplifying dependency management and version control.
- Deployed and maintained **AWS-based infrastructure** (EC2, Lambda, S3, CloudWatch), providing secure, scalable, and fault-tolerant environments for high-traffic enterprise workloads.
- Modeled **NoSQL data structures in MongoDB and DynamoDB**, leveraging partitioning and sharding for horizontal scalability and high availability.
- Established **asynchronous microservice communication** using **Apache Kafka and RabbitMQ**, enabling decoupled, event-driven workflows across core systems.
- Leveraged **Java advanced features** such as Streams, Lambda expressions, and Multithreading to simplify business logic and enhance backend performance for data-intensive workflows.
- Developed robust **persistence layers** using **Hibernate ORM**, implementing DAO patterns and optimized HQL/Criteria

queries for efficient data access and caching strategies.

- Practiced **Test-Driven Development (TDD)** using **JUnit**, **Mockito**, **Spock**, and **EasyMock**, achieving consistent code quality and reducing production defects through early validation.
- Actively participated in **Agile sprints and Scrum ceremonies**, contributing to iterative product releases, resolving technical debt, and aligning deliverables with evolving business objectives.
- Built responsive **Angular forms** using both Template-Driven and Reactive Forms approaches, incorporating real-time validation and improving user experience consistency.
- Managed **centralized configuration** using **Spring Config Server** and implemented unified logging with **Splunk** and **Log4j** for faster root cause analysis and improved observability.
- Applied **responsive design principles** with **Bootstrap**, **CSS3**, and **media queries**, ensuring cross-platform compatibility and seamless behavior on desktops, tablets, and mobile devices.
- Implemented **MyBatis with Spring MVC** for mapping complex SQL queries and optimizing ORM behavior for performance-sensitive enterprise use cases.
- Refactored **XML/XMI-based front-end logic** into **Angular component-driven architecture**, reducing code complexity by 60% and improving rendering performance by 45%.

TECHNICAL SKILLS:

Category	Technologies
Programming Languages	Java, J2EE, J2SE, JavaScript, TypeScript, SQL, PL/SQL, HTML5, CSS3, Shell Script, XML, JSON
Frameworks & Libraries	Spring Cloud, Netflix OSS (Eureka, Zuul, Ribbon, Hystrix), Gradle, Spring Boot, Spring MVC, Spring Security, Spring AOP, Spring DAO, Spring JDBC, Hibernate, JPA, MyBatis, Angular, React.js, Redux, Vue.js, Bootstrap, jQuery
Web Services & APIs	REST, SOAP, Swagger2, Swagger UI, Postman, SoapUI, JAX-RS, JAX-WS, JAX-RPC, EWS Library
Build Tools	Maven, Ant, Gradle
Testing Tools	JUnit, Mockito, Spock, EasyMock, Jasmine, Protractor, Karma, JMeter
Databases	Oracle, MongoDB, NoSQL, SQL Server, Redis
Cloud Platforms	AWS (EC2, S3, Lambda, RDS, CloudWatch), Pivotal Cloud Foundry (PCF)
CI/CD & DevOps	Jenkins, Docker, Concourse CI, AWS deployment Services
Servers	Apache Tomcat, JBoss, WebSphere
Messaging & Streaming	Apache Kafka, RabbitMQ, JMS, Apache Camel
Version Control	Git, GitHub, Bitbucket, SourceTree, JIRA
IDEs & Editors	Eclipse, IntelliJ IDEA, NetBeans, VS Code, Spring Tool Suite (STS)
Monitoring & Logging	Log4j, Splunk
Design & Patterns	DAO, DTO, MVC, Factory Pattern, Dependency Injection (IoC), OOAD, OOP, 12-Factor App
UI/UX Tools	AJAX, Firebug, CSS Blueprint, Dreamweaver, FrontPage
Testing Methodologies	TDD (Test-Driven Development), Unit Testing, Integration Testing

Organizational Experience

Client: Capital One Richmond, VA

Oct 2023 to Till Date

Role: Java Full Stack Developer

Project Responsibilities:

- Designed and developed highly interactive UI screens using **React.js**, incorporating **Redux** for state management and **NPM** libraries for component efficiency, resulting in seamless user experiences.
- Delivered **financial modeling and portfolio risk analysis modules** leveraging **Java 8+**, **Spring Boot**, and **ReactJS**, improving data accuracy and decision turnaround.
- Built robust **Single Page Applications (SPAs)** with **React Router**, optimizing performance and enabling fast navigation across dynamic views without full page reloads. Developed scalable and reusable front-end components, including data-driven tables in **ReactJS**, integrated with RESTful APIs to ensure consistent, real-time data rendering.
- Created a modern web application architecture by pairing **ReactJS** on the client-side with **Spring Boot** services on the backend, ensuring separation of concerns and scalability.
- Engineered responsive and mobile-friendly interfaces using **ReactJS** and **Bootstrap**, leveraging the virtual DOM for efficient updates based on backend API responses.
- Extended SPA capabilities with **Vue.js**, building modular components, and incorporating **Vue Router** to handle complex navigational flows across application modules.
- Built performant **RESTful APIs** using **Spring Boot** and **Spring Data JPA**, enabling reliable data exchange between front-end and back-end layers.
- Implemented optimized data processing logic using **Java Streams**, **Multithreading**, and **Lambda expressions** to handle large-scale financial data efficiently.
- Refactored legacy SOAP-based services to modern **Spring REST APIs** using **Spring Boot**, improving maintainability and aligning with REST architectural standards.
- Designed **Java REST APIs and event-driven workflows** with Spring Boot and Kafka to reduce p95 latency and improve throughput during high-concurrency market events.
- Deployed **Java services on AWS and internal Tomcat clusters**, standardizing health checks, log formats, and rollout strategies across hybrid infrastructure.
- Built **CI/CD with Jenkins/Gradle implementing DevSecOps checks** (unit, component, functional, performance, security), enforcing quality gates before promotion.
- Created **automation harnesses with JUnit, RestAssured, and WireMock**, virtualizing upstream services to unblock parallel test execution and improve coverage.
- Integrated **Oracle (PL/SQL) with DynamoDB** through resilient data access layers, optimizing query plans and partition keys for consistent, low-latency reads.
- **Produced architecture artifacts** (UML, ADRs, sequence diagrams) and verified that production code satisfied performance, security, and operability constraints.
- Defined and documented REST endpoints with **Swagger2**, accelerating integration for client teams and reducing back-and-forth in API communication.
- Deployed static frontend code to **AWS S3**, with proper **CORS** setup to securely consume APIs hosted in different domains.
- Streamlined microservice development by adopting **Spring Boot's** annotation-driven configuration and **bean profiling**.
- Integrated into REST APIs, ensuring encrypted communication and access control at the resource level.
- Applied **Spring Cloud Netflix OSS** tools like **Eureka**, **Zuul**, **Ribbon**, and **Hystrix** for dynamic service discovery, API gateway routing, and fault-tolerance.
- Enabled asynchronous process through **Apache Kafka**, enhancing message-driven communication between microservices.
- Designed NoSQL data models using **MongoDB**, including sharding and replica set configurations for scalable storage.
- Conducted live API testing using **Swagger UI**, enabling rapid validation during development and streamlining QA collaboration.
- **Automated Unit, Component, Functional, Performance, and Security tests** in Jenkins pipelines (JUnit, Mockito, JMeter, SAST/SCA), shrinking manual QA time and raising release confidence.
- Managed project lifecycle and dependency management with **Maven**, maintaining consistent builds and reducing manual overhead. Used **Docker** to containerize microservices, facilitating consistent deployment across dev, QA, and production environments.
- Automated build and deployment workflows using **Jenkins**, incorporating quality gates and parallel job execution.

- Applied **Gradle and Jenkins** for automated builds and test coverage, reducing release cycle time.
- Integrated **AWS Lambda, EC2, and S3** to orchestrate backend services and connect serverless functions with data persistence layers.
- Developed platform-agnostic APIs returning both **XML** and **JSON**, enabling use by mobile apps, partner portals, and browser-based clients.
- Integrated **TensorFlow Lite models** for on-device inference, enabling intelligent user behavior predictions and product recommendations within the frontend application, enhancing UX with real-time personalization.
- Deployed pre-trained **Hugging Face transformer models** (via REST APIs) for semantic search and text classification modules, augmenting backend services with **NLP** capabilities that improved content relevance and data tagging accuracy. Collaborated in Agile sprints to fix defects, address technical debts, and ensure continuous improvement of investment application performance.

Environment: RJava 8+, Spring Boot, Spring Cloud, Netflix OSS (Eureka, Ribbon, Zuul, Hystrix), Spring Data JPA, Hibernate, RESTful APIs, SOAP, Jersey, Swagger2, React.js, Redux, React Router, Angular, Vue.js, Bootstrap, Java Collections, Stream API, Multithreading, Kafka, RabbitMQ, Docker, Jenkins, Gradle, Git, GitHub, AWS (EC2, S3, Lambda, CloudWatch).

Client: Toyota India

June 2020 to July 2023

Role: Java Full Stack Developer

Project Responsibilities:

- Generated interactive UI templates using **JSP, HTML5, NodeJS, JavaScript, Ajax, Bootstrap**, and **CSS**, resulting in consistent cross-browser interfaces.
- Built and deployed **microservices for insurance product operations** using **Spring Boot, Spring Cloud, and AWS**, improving policy data synchronization.
- Developed multiple **Angular** components compatible with the latest **TypeScript** standards and **Angular CLI**, enhancing component reusability and maintainability.
- Hosted **Java APIs on Tomcat** for internal consumers while running microservices on AWS ECS, aligning configs and observability across on-prem and cloud.
- Implemented **service-oriented REST APIs** using Spring Boot (and **Dropwizard-style resource patterns** where applicable) to standardize contracts and error handling.
- Owned **DevSecOps automation** in Jenkins/GitLab CI—Unit, Component, Functional, Performance, and Security testing as mandatory stages before deployment.
- Introduced **service virtualization with Mockito/WireMock** and **API validation with RestAssured**, enabling parallel test execution and reducing integration wait states.
- Optimized **Oracle PL/SQL procedures and integrated with DynamoDB** access patterns to support telemetry and claims workflows with predictable performance.
- Leveraged the **MVC architecture** in Angular to streamline development and testing, improving code modularity and debugging efficiency.
- Migrated UI screens from **XMI (XML Metadata Interchange)** framework to **Angular**, significantly reducing code complexity and improving performance.
- Implemented **Angular Router** for seamless in-app navigation, improving the overall user experience and reducing page reload times.
- **Containerized Java services with Docker** and standardized image layers to reduce startup time and increase deployment reliability across environments.
- Automated **back-end integration and performance testing** using **JUnit/RestAssured/JMeter**, stabilizing SLAs under peak claim loads.
- Executed dynamic SQL queries within Java for CRUD operations tailored to complex business requirements, maintaining data integrity.
- Designed and implemented core **Java interfaces** and **abstract classes** to support scalable and modular application functionality.
- Transformed a monolithic application into a **Spring Boot** microservices architecture following **12-factor app** principles; deployed and scaled services using **PCF**, with complete manifest configuration.
- Built and deployed **RESTful microservices** using Spring Boot, integrating **Spring AOP** and **Spring Actuator** for

method-level metrics and performance tracking.

- Configured centralized property management using **Spring Config Server** and integrated **Splunk** for unified logging and observability. Automated deployment pipelines using **Concourse CI** and **Jenkins**, ensuring reliable and repeatable microservices deployments.
- Enhanced **fault-tolerance and service discovery** using **Netflix OSS (Eureka, Ribbon, Hystrix)** to support distributed microservices.
- Designed scalable, **high-performance financial systems** leveraging **Spring Boot, Hibernate, and RESTful APIs** for cross-service data flow. Developed REST services using the **EWS Library** to handle appointment creation, deletion, and updates in **Microsoft Outlook**.
- Implemented complete business functionality using **Spring Boot, Hibernate ORM, Spring MVC, Spring IOC, Spring Transaction, and Spring Security** to ensure security and data consistency. • Coded modular business rules and workflows within Java modules using **Spring Boot**, adhering to service-driven design.
- Applied **Spring AOP** and **Spring Batch** for transaction management and batch processing operations within distributed systems.
- Utilized the **Spring MVC framework** to enforce key design patterns like **IOC, DAO, DTO**, and ORM mappings for cleaner application architecture.
- Built scalable applications using **Spring, Hibernate annotations, and RESTful Web Services**, along with integration of **XML APIs** and supportive tools.
- Created complex **Hibernate POJOs** and managed persistence configurations to support robust ORM capabilities.
- Implemented secure authentication workflows using **OAuth2** and **OpenID Connect**, aligning with enterprise security standards.
- Integrated messaging systems using **Apache Camel** and **RabbitMQ** to enable asynchronous data flow across decoupled services.
- Configured and maintained **JBoss Application Server** alongside **Oracle** database integration to support scalable enterprise applications. Designed, developed, and tested enterprise APIs using **SoapUI** and **Postman**, ensuring functional and performance benchmarks were met.
- Conducted comprehensive unit and use-case integration testing using **JUnit**, guaranteeing code reliability and deployment. Configured **Netflix OSS components (Eureka, Zuul, Hystrix)** to achieve fault-tolerant and discoverable microservices.
- Automated **unit and integration tests** using JUnit and Mockito, full test coverage for core insurance workflows.
- Leveraged **Log4j** for configurable logging and debugging at various severity levels, aiding in root-cause analysis and system monitoring.
- Utilized **AWS deployment services** to configure and launch scalable web environments and virtual servers rapidly.
- Developed **JSON** responses for client-side AJAX requests using **GET** and **POST**, supporting dynamic data updates and seamless interactions.
- Collaborated with data scientists to integrate a fraud detection microservice powered by an in-house **PyTorch** model, exposing inference via secured **REST APIs** and enabling near real-time decisioning in insurance quote workflows.
- Implemented a recommendation engine using **Scikit-learn** to dynamically suggest relevant health plans based on user demographics and search history, improving engagement metrics by 20% across **Angular** and **Spring Boot** modules.

Environments: Java 8+, Spring Boot, Spring Cloud, Netflix OSS (Eureka, Ribbon, Zuul, Hystrix), Spring MVC, Spring Security, Spring AOP, Spring Batch, Spring Config Server, Hibernate ORM, Spring Data JPA, RESTful APIs, SOAP, Angular, Angular CLI, TypeScript, React.js, Bootstrap, HTML5, CSS3, JavaScript, Ajax, JSON, XML, Apache Kafka, RabbitMQ, Docker, Jenkins, Gradle, Git, GitHub, Concourse CI, Maven, PCF (Pivotal Cloud Foundry), AWS (EC2, S3, RDS, CloudWatch), Oracle, SQL Server, Redis, MongoDB, OAuth2, OpenID Connect, JUnit, Mockito, Jasmine, Protractor, JMeter, SoapUI, Postman, Log4j, Splunk, Swagger2.

Client: PNB MetLife Insurance India

Sep 2018 – May 2020

Role: Java UI Developer

Project Responsibilities:

- Applied **Agile methodologies** and **Test-Driven Development (TDD)** to deliver iterative, production-ready software aligned with evolving business needs.
- Designed scalable systems using **Object-Oriented Analysis and Design (OOAD)** and **J2EE design patterns**, ensuring high cohesion and low coupling across modules.
- Implemented the **Model-View-Controller (MVC)** pattern using **Spring** and **Angular**, facilitating a clean separation of

concerns between data, logic, and presentation layers.

- Enhanced the portal user interface using **HTML**, **CSS**, **JavaScript**, and **XML**, integrating both client-side validation and server-side **Bean Validation Framework** for robust input handling.
- Launched the front-end using **React Class Components** and managed component states effectively, enabling smooth and modular UI updates.
- Integrated advanced **ReactJS** functionalities like form validation, CRUD operations, pagination, filtering, multi-selection, custom directives, and authentication features, resulting in highly interactive web pages.
- Designed microservice-based architecture using **Spring Boot** and **Amazon Web Services (AWS)**, implementing messaging through **REST** and **MQ** for identity and access microservices. Built resilient REST API endpoints using **Spring Boot** and incorporated **messaging queues** to support asynchronous, continuous delivery pipelines.
- Deployed secured microservices on **AWS Cloud** using **Spring Security**, enabling encrypted access and efficient user authorization.
- Utilized **Spring MVC** in the presentation layer for user flow control and validations, transferring data seamlessly via **state props** in ReactJS.
- Created interactive pages using **JavaScript**, **jQuery**, and **Bootstrap**, optimized with **media queries** for responsive design across devices.
- Developed and maintained unit test cases using **JUnit**, **Mockito**, and **EasyMock**, ensuring robust and regression-proof features. Performed load and stress testing with **JMeter**, created scripts for simulation, and reports to identify performance bottlenecks.
- Used **Apache Maven** for lifecycle management and build automation, integrated with **Bitbucket** for version control, and team collaboration.
- Implemented detailed logging and diagnostics using **Log4j**, supporting early detection and resolution of application issues.

Environments: Object Oriented Concepts, OOAD, JMeter, Apache Maven, log4j, Bitbucket, JUnit, Mockito, Easy mock, Angular JS, Spring Boot, HTML, JavaScript, XML, Java, React Class Component.